# LibanPost eShop - database schema

# Contents


Table of contents . . . 4
Addresses . . . 5
AuditLogs . . . 5
CartItems . . . 5
Carts . . . 6
Categories . . . 6
Coupons . . . 7
CouponUsages . . . 7
EmailMessages . . . 8
Log . . . 8
OrderLines . . . 9
Orders . . . 9
OrderShipments . . . 11
PaymentEvents . . . 11
Payments . . . 12
ProductImages . . . 12
Products . . . 13
ProductVariants . . . 13
ReturnLines . . . 14
ReturnRequests . . . 14
Reviews . . . 15
ReviewVotes . . . 16
ShipmentEvents . . . 16
ShopListings . . . 16
Shops . . . 17
ShopUsers . . . 18
StockMovements . . . 19
TaxClasses . . . 19
UserRoles . . . 19
Users . . . 20
VendorLedgerEntries . . . 20
VendorPayouts . . . 21
WishlistItems . . . 22
Relationship overview . . . 22
- Identity and accounts . . . 22
- Catalogue . . . 22
- Orders and fulfilment . . . 22

Returns (RMA) . . . 22
Reviews . . . 23
Marketplace . . . 23
Vendor money . . . 23
Messaging and observability . . . 23
Key data flows . . . 23
1. Purchase (cart to paid order) . . . 23
2. Return (RMA) with its refund . . . 23
3. Settlement (paying a vendor) . . . 24
4. Notifications (outbox, not SMTP) . . . 24
Status and enum columns . . . 24
What is derived, what is stamped . . . 25
Uniqueness and enforcement of the rules . . . 25
Design notes . . . 26
Data volume and analysis (live demo instance) . . . 26
Observations . . . 27
Where the weight is . . . 27

This document describes the SQL Server database (`MVStores`) that backs the application: every table, its columns, primary keys, foreign keys and the relationships between them. Column types and nullability are taken from the live database (`INFORMATION_SCHEMA`); lengths are character lengths for string columns.

**Conventions**

- `Id` (int, identity) is the primary key of every table unless noted.
- `CreatedUtc` / `UpdatedUtc` are UTC timestamps; `*Utc` columns are UTC.
- Money is stored as `decimal(18,2)`.
- Foreign keys use the SQL Server default `ON DELETE` behaviour shown per constraint

(`CASCADE`, `SET NULL`, `NO ACTION`).

- The application also creates guarded, idempotent DDL at start-up for a few tables/columns

(see `Data/*Schema.cs`); the list below reflects the resulting schema.

## Table of contents


- [Addresses](#addresses)
- [AuditLogs](#auditlogs)
- [CartItems](#cartitems)
- [Carts](#carts)
- [Categories](#categories)
- [Coupons](#coupons)
- [CouponUsages](#couponusages)
- [EmailMessages](#emailmessages)
- [Log](#log)
- [OrderLines](#orderlines)
- [Orders](#orders)
- [OrderShipments](#ordershipments)
- [PaymentEvents](#paymentevents)
- [Payments](#payments)
- [ProductImages](#productimages)
- [Products](#products)
- [ProductVariants](#productvariants)
- [ReturnLines](#returnlines)
- [ReturnRequests](#returnrequests)
- [Reviews](#reviews)
- [ReviewVotes](#reviewvotes)
- [ShipmentEvents](#shipmentevents)
- [ShopListings](#shoplistings)
- [Shops](#shops)
- [ShopUsers](#shopusers)
- [StockMovements](#stockmovements)
- [TaxClasses](#taxclasses)
- [UserRoles](#userroles)
- [Users](#users)
- [VendorLedgerEntries](#vendorledgerentries)
- [VendorPayouts](#vendorpayouts)
- [WishlistItems](#wishlistitems)

## Addresses

Saved delivery addresses for an account. Chosen at checkout and reused for the next order; removed with the account (CASCADE).

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| `UserId` | `Users.Id` | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |
| `UserId` | nvarchar(450) | NO | — |
| `FullName` | nvarchar(120) | NO | — |
| `Line1` | nvarchar(200) | NO | — |
| `Line2` | nvarchar(200) | YES | — |
| `City` | nvarchar(100) | NO | — |
| `State` | nvarchar(100) | YES | — |
| `PostalCode` | nvarchar(20) | NO | — |
| `Country` | nvarchar(80) | NO | — |
| `Phone` | nvarchar(40) | YES | — |
| `IsDefault` | bit | NO | — |
| `CreatedUtc` | datetime2 | NO | — |

## AuditLogs

The administrative trail: who performed which staff action, on which record, when, and a one-sentence summary. References are deliberately loose (entity type + id, actor display name) so history survives deletes; writes are best-effort and never block the action they document.

**Primary key:** `Id`

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |
| `Actor` | nvarchar(256) | NO | — |
| `Action` | nvarchar(80) | NO | — |
| `EntityType` | nvarchar(40) | NO | — |
| `EntityId` | int | YES | — |
| `Summary` | nvarchar(500) | YES | — |
| `CreatedUtc` | datetime2 | NO | — |

## CartItems

One row per bought unit in a cart: the variant, the quantity and the unit price captured when the item was added.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| `CartId` | `Carts.Id` | CASCADE |

| Column | References | On delete |
|---|---|---|
| ProductVariantId | ProductVariants.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| CartId | int | NO | — |
| ProductVariantId | int | NO | — |
| Quantity | int | NO | — |
| AddedUtc | datetime2 | NO | — |

## Carts

An open basket. Signed-in shoppers own exactly one (UserId); guest baskets carry AnonymousId and attach to the account at sign-in.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| UserId | Users.Id | SET_NULL |

**Referenced by (parent of):**

- `CartItems.CartId` -> `Carts.Id` (CASCADE)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| UserId | nvarchar(450) | YES | — |
| AnonymousId | nvarchar(64) | YES | — |
| CreatedUtc | datetime2 | NO | — |
| UpdatedUtc | datetime2 | NO | — |

## Categories

The product taxonomy. A category may nest under another (ParentId) and is either global or scoped to one shop (ShopId); Slug is unique per shop.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ParentId | Categories.Id | NO_ACTION |
| ShopId | Shops.Id | CASCADE |

**Referenced by (parent of):**

- `Categories.ParentId` -> `Categories.Id` (NO_ACTION)
- `Products.CategoryId` -> `Categories.Id` (NO_ACTION)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ShopId | int | YES | — |
| IsGlobal | bit | NO | — |
| Name | nvarchar(80) | NO | — |

| Column | Type | Nullable | Default |
|---|---|---|---|
| Slug | nvarchar(80) | NO | — |
| Description | nvarchar(500) | YES | — |
| ParentId | int | YES | — |
| DisplayOrder | int | NO | — |
| IsActive | bit | NO | — |

## Coupons

Discount-code rules: percent or fixed amount, minimum spend, validity window, total and per-shopper use caps, optional single-shop scoping, activate toggle. Redeemed codes are deactivated, never deleted.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ShopId | Shops.Id | NO_ACTION |

**Referenced by (parent of):**

- `CouponUsages.CouponId` -> `Coupons.Id` (CASCADE)
- `Orders.CouponId` -> `Coupons.Id` (SET_NULL)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| Code | nvarchar(64) | NO | — |
| Description | nvarchar(256) | YES | — |
| Type | int | NO | — |
| Value | decimal | NO | — |
| MinSubtotal | decimal | NO | — |
| MaxUses | int | YES | — |
| MaxUsesPerUser | int | YES | — |
| UsedCount | int | NO | — |
| StartsUtc | datetime2 | YES | — |
| ExpiresUtc | datetime2 | YES | — |
| IsActive | bit | NO | — |
| AppliesToAll | bit | NO | — |
| ShopId | int | YES | — |
| CreatedUtc | datetime2 | NO | — |

## CouponUsages

The redemption trail: one row per (coupon, shopper, order), so who used what when is replayable and the use caps are re-checkable at order time.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| CouponId | Coupons.Id | CASCADE |
| OrderId | Orders.Id | SET_NULL |

| Column | References | On delete |
|---|---|---|
| UserId | Users.Id | SET_NULL |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| CouponId | int | NO | — |
| UserId | nvarchar(450) | YES | — |
| OrderId | int | YES | — |
| RedeemedUtc | datetime2 | NO | — |

## EmailMessages

The transactional outbox. Nothing sends on the request path: a checkout queues a row and a background dispatcher delivers it. Carry a DedupeKey (a retried request cannot mail twice), an attempt counter with backoff, and Skipped/Failed terminal states. Loose references (UserId/OrderId) column for the preview page.

**Primary key:** Id

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ToAddress | nvarchar(256) | NO | — |
| ToName | nvarchar(120) | YES | — |
| Subject | nvarchar(200) | NO | — |
| Kind | nvarchar(60) | NO | — |
| BodyHtml | nvarchar(max) | NO | — |
| BodyText | nvarchar(max) | NO | — |
| Status | int | NO | — |
| Attempts | int | NO | — |
| LastError | nvarchar(1000) | YES | — |
| DedupeKey | nvarchar(120) | YES | — |
| UserId | nvarchar(450) | YES | — |
| OrderId | int | YES | — |
| CreatedUtc | datetime2 | NO | — |
| SentUtc | datetime2 | YES | — |
| NextAttemptUtc | datetime2 | YES | — |

## Log

The application's database log sink: level, message, logger, optional exception, UTC timestamp. Batched by a background writer; retention is optional.

**Primary key:** Id

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | bigint | NO | — |
| Application | nvarchar(50) | NO | — |
| Logged | datetime | NO | — |
| Level | nvarchar(50) | NO | — |
| Message | nvarchar(max) | NO | — |
| Logger | nvarchar(250) | YES | — |

| Column | Type | Nullable | Default |
|---|---|---|---|
| Callsite | nvarchar(max) | YES | — |
| Exception | nvarchar(max) | YES | — |

## OrderLines

A full snapshot of each sold line at purchase time: product/variant names, SKU, unit price, quantity, line totals. Editing or deleting catalog rows can never rewrite an order's history.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| OrderId | Orders.Id | CASCADE |
| ProductVariantId | ProductVariants.Id | SET_NULL |

**Referenced by (parent of):**

- `ReturnLines.OrderLineId` -> `OrderLines.Id` (NO_ACTION)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| OrderId | int | NO | — |
| ProductVariantId | int | YES | — |
| ProductId | int | YES | — |
| ProductName | nvarchar(160) | NO | — |
| VariantName | nvarchar(80) | NO | — |
| Sku | nvarchar(64) | NO | — |
| ImageUrl | nvarchar(max) | YES | — |
| Quantity | int | NO | — |
| UnitPrice | decimal | NO | — |
| LineTotal | decimal | NO | — |

## Orders

The order document and its state machine: money totals (subtotal, discount, shipping, tax, grand total, refunded running total), payment method and collection status, addresses, coupon used, and the IdempotencyKey that makes a double-clicked checkout harmless. Status transitions are validated in OrderService.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| CouponId | Coupons.Id | SET_NULL |
| ShopId | Shops.Id | NO_ACTION |
| UserId | Users.Id | SET_NULL |

**Referenced by (parent of):**

- `CouponUsages.OrderId` -> `Orders.Id` (SET_NULL)
- `OrderLines.OrderId` -> `Orders.Id` (CASCADE)
- `OrderShipments.OrderId` -> `Orders.Id` (CASCADE)
- `PaymentEvents.OrderId` -> `Orders.Id` (SET_NULL)

- `Payments.OrderId` -> `Orders.Id` (CASCADE)
- `VendorLedgerEntries.OrderId` -> `Orders.Id` (SET_NULL)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |
| `ShopId` | int | NO | — |
| `OrderNumber` | nvarchar(32) | NO | — |
| `UserId` | nvarchar(450) | YES | — |
| `Email` | nvarchar(256) | NO | — |
| `CustomerName` | nvarchar(120) | NO | — |
| `Phone` | nvarchar(40) | YES | — |
| `Status` | int | NO | — |
| `PaymentMethod` | int | NO | — |
| `CollectionStatus` | int | NO | — |
| `CodAmount` | decimal | NO | — |
| `IdempotencyKey` | nvarchar(64) | YES | — |
| `Subtotal` | decimal | NO | — |
| `ShippingTotal` | decimal | NO | — |
| `TaxTotal` | decimal | NO | — |
| `DiscountTotal` | decimal | NO | — |
| `GrandTotal` | decimal | NO | — |
| `ShippingName` | nvarchar(120) | NO | — |
| `ShippingLine1` | nvarchar(200) | NO | — |
| `ShippingLine2` | nvarchar(200) | YES | — |
| `ShippingCity` | nvarchar(100) | NO | — |
| `ShippingState` | nvarchar(100) | YES | — |
| `ShippingPostalCode` | nvarchar(20) | NO | — |
| `ShippingCountry` | nvarchar(80) | NO | — |
| `CustomerNote` | nvarchar(1000) | YES | — |
| `PlacedUtc` | datetime2 | NO | — |
| `PaidUtc` | datetime2 | YES | — |
| `ShippedUtc` | datetime2 | YES | — |
| `DeliveredUtc` | datetime2 | YES | — |
| `CancelledUtc` | datetime2 | YES | — |
| `TrackingNumber` | nvarchar(64) | YES | — |
| `Carrier` | nvarchar(40) | YES | — |
| `CarrierShipmentId` | nvarchar(120) | YES | — |
| `LabelUrl` | nvarchar(max) | YES | — |
| `CarrierError` | nvarchar(500) | YES | — |
| `CouponCode` | nvarchar(64) | YES | — |
| `CouponId` | int | YES | — |
| `RefundedTotal` | decimal | NO | ((0)) |

## OrderShipments

The consignment handed to a carrier: carrier, tracking number, status, per-shop filtering for the vendor order book.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| `OrderId` | `Orders.Id` | CASCADE |
| `ShopId` | `Shops.Id` | NO_ACTION |

**Referenced by (parent of):**

- `ShipmentEvents.ShipmentId` -> `OrderShipments.Id` (CASCADE)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |
| `OrderId` | int | NO | — |
| `ShopId` | int | NO | — |
| `Carrier` | nvarchar(40) | NO | — |
| `CarrierShipmentId` | nvarchar(120) | YES | — |
| `TrackingNumber` | nvarchar(120) | YES | — |
| `LabelUrl` | nvarchar(max) | YES | — |
| `Status` | int | NO | — |
| `WeightGrams` | int | NO | — |
| `ShippingCost` | decimal | NO | — |
| `CodAmount` | decimal | NO | — |
| `RecipientName` | nvarchar(120) | YES | — |
| `RecipientPhone` | nvarchar(40) | YES | — |
| `DeliveryAddress` | nvarchar(400) | YES | — |
| `PickupLocation` | nvarchar(120) | YES | — |
| `LastError` | nvarchar(600) | YES | — |
| `CreatedUtc` | datetime2 | NO | — |
| `DispatchedUtc` | datetime2 | YES | — |
| `DeliveredUtc` | datetime2 | YES | — |

## PaymentEvents

Raw signed events received from the card gateway (the simulator signs exactly as a real provider would). Unique per (Provider, EventId), so a redelivered webhook is stored once; the source of truth for capture decisions.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| `OrderId` | `Orders.Id` | SET_NULL |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |

| Column | Type | Nullable | Default |
|---|---|---|---|
| Provider | nvarchar(40) | NO | — |
| EventId | nvarchar(120) | NO | — |
| Type | nvarchar(60) | NO | — |
| OrderId | int | YES | — |
| OrderNumber | nvarchar(32) | YES | — |
| TransactionId | nvarchar(120) | YES | — |
| Amount | decimal | NO | — |
| Currency | nvarchar(8) | NO | — |
| Payload | nvarchar(max) | NO | — |
| SignatureValid | bit | NO | — |
| Outcome | nvarchar(40) | NO | — |
| Note | nvarchar(500) | YES | — |
| CreatedUtc | datetime2 | NO | — |

## Payments

Money-movement records per order: charge attempts, refunds, and their issuing GatewayReference (e.g. simulator charge id sbr_...). Cash-on-delivery refunds are bookkeeping rows issued by staff.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| OrderId | Orders.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| OrderId | int | NO | — |
| Provider | nvarchar(40) | NO | — |
| TransactionId | nvarchar(120) | YES | — |
| Amount | decimal | NO | — |
| Currency | nvarchar(8) | NO | — |
| Status | int | NO | — |
| FailureReason | nvarchar(500) | YES | — |
| IsTest | bit | NO | — |
| CreatedUtc | datetime2 | NO | — |
| CompletedUtc | datetime2 | YES | — |

## ProductImages

The gallery of one product: either a typed URL or an uploaded file stored by the local image store. First image is primary; ordering is explicit.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ProductId | Products.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ProductId | int | NO | — |
| Url | nvarchar(max) | NO | — |
| AltText | nvarchar(200) | YES | — |
| DisplayOrder | int | NO | — |
| IsPrimary | bit | NO | — |

## Products

A catalogue item owned by a shop: name, slug (unique per shop), brand, descriptions, publish state, soft-delete flag, category and tax class, rating snapshot columns for fast sorting.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| CategoryId | Categories.Id | NO_ACTION |
| ShopId | Shops.Id | CASCADE |
| TaxClassId | TaxClasses.Id | SET_NULL |

**Referenced by (parent of):**

- ProductImages.ProductId -> Products.Id (CASCADE)
- ProductVariants.ProductId -> Products.Id (CASCADE)
- Reviews.ProductId -> Products.Id (NO_ACTION)
- ShopListings.ProductId -> Products.Id (NO_ACTION)
- WishlistItems.ProductId -> Products.Id (CASCADE)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ShopId | int | NO | — |
| Name | nvarchar(160) | NO | — |
| Slug | nvarchar(160) | NO | — |
| ShortDescription | nvarchar(200) | YES | — |
| Description | nvarchar(max) | YES | — |
| Brand | nvarchar(120) | YES | — |
| CategoryId | int | NO | — |
| IsPublished | bit | NO | — |
| IsDeleted | bit | NO | — |
| IsFeatured | bit | NO | — |
| CreatedUtc | datetime2 | NO | — |
| UpdatedUtc | datetime2 | NO | — |
| TaxClassId | int | YES | — |

## ProductVariants

A buyable SKU of a product: variant name, SKU (unique store-wide), stock-on-hand, price (falls back to the product price when null). Cart, orders, listings and stock movements all point here.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| `ProductId` | `Products.Id` | CASCADE |

**Referenced by (parent of):**

- `CartItems.ProductVariantId` -> `ProductVariants.Id` (CASCADE)
- `OrderLines.ProductVariantId` -> `ProductVariants.Id` (SET_NULL)
- `ShopListings.ProductVariantId` -> `ProductVariants.Id` (SET_NULL)
- `StockMovements.ProductVariantId` -> `ProductVariants.Id` (CASCADE)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |
| `ProductId` | int | NO | — |
| `Sku` | nvarchar(64) | NO | — |
| `Name` | nvarchar(80) | NO | — |
| `Price` | decimal | NO | — |
| `CompareAtPrice` | decimal | YES | — |
| `StockQuantity` | int | NO | — |
| `IsActive` | bit | NO | — |
| `LowStockThreshold` | int | NO | — |
| `RowVersion` | varbinary | NO | — |

## ReturnLines

Per-line units of a return: which order line, how many units were sent back. The refund amount is recomputed from these rows, the order discount and the tax rate.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| `OrderLineId` | `OrderLines.Id` | NO_ACTION |
| `ReturnRequestId` | `ReturnRequests.Id` | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |
| `ReturnRequestId` | int | NO | — |
| `OrderLineId` | int | NO | — |
| `Quantity` | int | NO | — |
| `UnitPrice` | decimal | NO | — |

## ReturnRequests

The RMA header: reference RMA-{year}-{n}, lifecycle (Requested -> Approved -> Received -> Refunded, with Rejected/Cancelled exits), requester, shop, order, decision/receipt/refund timestamps, the computed RefundAmount, and both sides' notes. One open request per order is enforced in the service.

**Primary key:** `Id`

**Referenced by (parent of):**

- `ReturnLines.ReturnRequestId` -> `ReturnRequests.Id` (CASCADE)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| OrderId | int | NO | — |
| ShopId | int | NO | — |
| Reference | nvarchar(40) | NO | — |
| Status | int | NO | — |
| UserId | nvarchar(450) | YES | — |
| Email | nvarchar(256) | NO | — |
| CustomerNote | nvarchar(1000) | YES | — |
| DecisionNote | nvarchar(1000) | YES | — |
| HandledBy | nvarchar(256) | YES | — |
| RefundAmount | decimal | NO | ((0)) |
| RequestedUtc | datetime2 | NO | (getutcdate()) |
| DecidedUtc | datetime2 | YES | — |
| ReceivedUtc | datetime2 | YES | — |
| RefundedUtc | datetime2 | YES | — |

## Reviews

A product review: rating, title/body, author, verified-purchase flag, moderation state, seller reply. Aggregates (averages, histograms, shop scores) are computed at query time from these rows.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ProductId | Products.Id | NO_ACTION |
| ShopId | Shops.Id | NO_ACTION |
| UserId | Users.Id | SET_NULL |

**Referenced by (parent of):**

- ReviewVotes.ReviewId -> Reviews.Id (CASCADE)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ProductId | int | NO | — |
| ShopId | int | NO | — |
| UserId | nvarchar(450) | YES | — |
| ReviewerName | nvarchar(120) | NO | — |
| Rating | int | NO | — |
| Title | nvarchar(140) | YES | — |
| Body | nvarchar(4000) | YES | — |
| IsVerifiedPurchase | bit | NO | — |
| Status | int | NO | — |
| HelpfulCount | int | NO | — |
| NotHelpfulCount | int | NO | — |
| VendorReply | nvarchar(2000) | YES | — |

| Column | Type | Nullable | Default |
|---|---|---|---|
| VendorReplyUtc | datetime2 | YES | — |
| CreatedUtc | datetime2 | NO | — |
| PublishedUtc | datetime2 | YES | — |
| UpdatedUtc | datetime2 | YES | — |

## ReviewVotes

Per-user helpful/unhelpful votes on reviews; one vote each, unique per (review, user).

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ReviewId | Reviews.Id | CASCADE |
| UserId | Users.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ReviewId | int | NO | — |
| UserId | nvarchar(450) | NO | — |
| IsHelpful | bit | NO | — |
| CreatedUtc | datetime2 | NO | — |

## ShipmentEvents

The carrier lifecycle timeline of a consignment (created, picked up, in transit, out for delivery, delivered, failed, returned, cancelled).

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ShipmentId | OrderShipments.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ShipmentId | int | NO | — |
| Status | int | NO | — |
| Location | nvarchar(80) | YES | — |
| Description | nvarchar(300) | YES | — |
| OccurredUtc | datetime2 | NO | — |
| RecordedUtc | datetime2 | NO | — |
| FromCarrier | bit | NO | — |

## ShopListings

A shop's own listing of a product variant, with the shop's own stock pool and active flag. Received returns restock here first, falling back to the variant's shared stock.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| `ProductId` | `Products.Id` | NO_ACTION |
| `ProductVariantId` | `ProductVariants.Id` | SET_NULL |
| `ShopId` | `Shops.Id` | NO_ACTION |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |
| `ShopId` | int | NO | — |
| `ProductId` | int | NO | — |
| `ProductVariantId` | int | YES | — |
| `Price` | decimal | YES | — |
| `CompareAtPrice` | decimal | YES | — |
| `StockQuantity` | int | NO | — |
| `LowStockThreshold` | int | NO | — |
| `IsActive` | bit | NO | — |
| `Sku` | nvarchar(64) | YES | — |
| `Note` | nvarchar(500) | YES | — |
| `RowVersion` | varbinary | NO | — |
| `CreatedUtc` | datetime2 | NO | — |
| `UpdatedUtc` | datetime2 | NO | — |

## Shops

Vendor storefronts: name, slug, description, theme, commission rate (fraction, goods only), lifecycle status (pending/approved/rejected/suspended/closed), the payout destination (account name, bank, IBAN) and an optional contact e-mail used for shop notifications.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| `OwnerUserId` | `Users.Id` | NO_ACTION |

**Referenced by (parent of):**

- `Categories.ShopId` -> `Shops.Id` (CASCADE)
- `Coupons.ShopId` -> `Shops.Id` (NO_ACTION)
- `Orders.ShopId` -> `Shops.Id` (NO_ACTION)
- `OrderShipments.ShopId` -> `Shops.Id` (NO_ACTION)
- `Products.ShopId` -> `Shops.Id` (CASCADE)
- `Reviews.ShopId` -> `Shops.Id` (NO_ACTION)
- `ShopListings.ShopId` -> `Shops.Id` (NO_ACTION)
- `ShopUsers.ShopId` -> `Shops.Id` (CASCADE)
- `VendorLedgerEntries.ShopId` -> `Shops.Id` (CASCADE)
- `VendorPayouts.ShopId` -> `Shops.Id` (CASCADE)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **`Id`** (PK) | int | NO | — |
| `OwnerUserId` | nvarchar(450) | NO | — |

| Column | Type | Nullable | Default |
|---|---|---|---|
| Name | nvarchar(120) | NO | — |
| Slug | nvarchar(120) | NO | — |
| Description | nvarchar(600) | YES | — |
| Tagline | nvarchar(160) | YES | — |
| Status | int | NO | — |
| StatusReason | nvarchar(500) | YES | — |
| ContactName | nvarchar(120) | YES | — |
| Phone | nvarchar(40) | YES | — |
| Email | nvarchar(256) | YES | — |
| Website | nvarchar(200) | YES | — |
| AddressLine1 | nvarchar(200) | YES | — |
| City | nvarchar(100) | YES | — |
| PostalCode | nvarchar(40) | YES | — |
| Country | nvarchar(80) | YES | — |
| PickupLocation | nvarchar(120) | YES | — |
| CommissionRate | decimal | NO | — |
| FlatShippingRate | decimal | YES | — |
| FreeShippingThreshold | decimal | YES | — |
| Currency | nvarchar(8) | NO | — |
| ThemeKey | nvarchar(40) | NO | — |
| LogoUrl | nvarchar(500) | YES | — |
| BannerUrl | nvarchar(500) | YES | — |
| IsFeatured | bit | NO | — |
| CreatedUtc | datetime2 | NO | — |
| ApprovedUtc | datetime2 | YES | — |
| SuspendedUtc | datetime2 | YES | — |
| BankAccountName | nvarchar(120) | YES | — |
| BankName | nvarchar(120) | YES | — |
| BankIban | nvarchar(40) | YES | — |

## ShopUsers

Shop-user membership: which account can act on which shop, and in what role (Owner, Manager, Fulfilment). Scoping checks use it so another shop's data is a 404 rather than a redacted view.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ShopId | Shops.Id | CASCADE |
| UserId | Users.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ShopId | int | NO | — |
| UserId | nvarchar(450) | NO | — |

| Column | Type | Nullable | Default |
|---|---|---|---|
| Role | int | NO | — |
| AcceptedTermsUtc | datetime2 | YES | — |
| CreatedUtc | datetime2 | NO | — |

## StockMovements

Append-only stock ledger: why a variant's stock changed (+/- quantity, reason, optional order link). Auditable and never edited after write.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ProductVariantId | ProductVariants.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ProductVariantId | int | NO | — |
| QuantityDelta | int | NO | — |
| StockAfter | int | NO | — |
| Reason | int | NO | — |
| Note | nvarchar(200) | YES | — |
| OrderId | int | YES | — |
| CreatedBy | nvarchar(100) | YES | — |
| CreatedUtc | datetime2 | NO | — |

## TaxClasses

Tax rate definitions applied by class to product lines; the default rate falls back to Store:TaxRate when none is set.

**Primary key:** `Id`

**Referenced by (parent of):**

- `Products.TaxClassId` -> `TaxClasses.Id` (SET_NULL)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| Name | nvarchar(80) | NO | — |
| Rate | decimal | NO | — |
| IsActive | bit | NO | — |
| IsExempt | bit | NO | — |
| CreatedUtc | datetime2 | NO | — |

## UserRoles

Role rows per account (User, Vendor, Admin) backed by cookie roles that gate the areas. Unique per (user, role).

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| UserId | Users.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| UserId | nvarchar(450) | NO | — |
| RoleName | nvarchar(40) | NO | — |

## Users

Accounts: e-mail (normalised unique index), password hash, display name, lockout/confirmation flags, and the audit-irrelevant convenience flags the demo needs. Deleted accounts release carts (SET NULL) while orders keep running totals.

**Primary key:** `Id`

**Referenced by (parent of):**

- `Addresses.UserId` -> `Users.Id` (CASCADE)
- `Carts.UserId` -> `Users.Id` (SET_NULL)
- `CouponUsages.UserId` -> `Users.Id` (SET_NULL)
- `Orders.UserId` -> `Users.Id` (SET_NULL)
- `Reviews.UserId` -> `Users.Id` (SET_NULL)
- `ReviewVotes.UserId` -> `Users.Id` (CASCADE)
- `Shops.OwnerUserId` -> `Users.Id` (NO_ACTION)
- `ShopUsers.UserId` -> `Users.Id` (CASCADE)
- `UserRoles.UserId` -> `Users.Id` (CASCADE)
- `WishlistItems.UserId` -> `Users.Id` (CASCADE)

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | nvarchar(450) | NO | — |
| Email | nvarchar(256) | NO | — |
| NormalizedEmail | nvarchar(256) | NO | — |
| DisplayName | nvarchar(120) | NO | — |
| Phone | nvarchar(40) | YES | — |
| PasswordHash | nvarchar(400) | NO | — |
| SecurityStamp | nvarchar(64) | NO | — |
| IsActive | bit | NO | — |
| AccessFailedCount | int | NO | — |
| LockoutEndUtc | datetime2 | YES | — |
| CreatedUtc | datetime2 | NO | — |
| LastLoginUtc | datetime2 | YES | — |

## VendorLedgerEntries

The vendor money ledger and single source of truth for what a shop is owed: Sale credits on collection, Reversals on cancellation/refund (ReturnRequestId column when a return caused it), manual Adjustments, and Payout rows; every row carries gross/commission/net and, once settled, the settling payout's id, reference and SettledUtc.

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| OrderId | Orders.Id | SET_NULL |
| ShopId | Shops.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ShopId | int | NO | — |
| OrderId | int | YES | — |
| Type | int | NO | — |
| GrossAmount | decimal | NO | — |
| CommissionAmount | decimal | NO | — |
| NetAmount | decimal | NO | — |
| Note | nvarchar(200) | YES | — |
| SettledUtc | datetime2 | YES | — |
| PayoutReference | nvarchar(120) | YES | — |
| CreatedUtc | datetime2 | NO | — |
| ReturnRequestId | int | YES | — |

## VendorPayouts

A settlement of a shop's outstanding balance: reference PAY-{year}-{n}, period, entry count and gross/commission/net totals, method and note. Lifecycle Paid -> Failed (bank refused; covered entries are released back) -> Reversed (recorded in error; same release), each stamped with who handled it and when.

**Primary key:** Id

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ShopId | Shops.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| ShopId | int | NO | — |
| Reference | nvarchar(40) | NO | — |
| PeriodStartUtc | datetime2 | NO | — |
| PeriodEndUtc | datetime2 | NO | — |
| EntryCount | int | NO | — |
| GrossAmount | decimal | NO | — |
| CommissionAmount | decimal | NO | — |
| NetAmount | decimal | NO | — |
| Method | nvarchar(40) | NO | — |
| Note | nvarchar(200) | YES | — |
| CreatedBy | nvarchar(256) | YES | — |
| CreatedUtc | datetime2 | NO | — |
| Status | int | NO | ((0)) |
| FailureReason | nvarchar(500) | YES | — |

| Column | Type | Nullable | Default |
|---|---|---|---|
| HandledBy | nvarchar(256) | YES | — |
| FailedUtc | datetime2 | YES | — |
| ReversedUtc | datetime2 | YES | — |

## WishlistItems

Saved products per shopper; unique per (user, product).

**Primary key:** `Id`

**Foreign keys (this table -> parent):**

| Column | References | On delete |
|---|---|---|
| ProductId | Products.Id | CASCADE |
| UserId | Users.Id | CASCADE |

| Column | Type | Nullable | Default |
|---|---|---|---|
| **Id** (PK) | int | NO | — |
| UserId | nvarchar(450) | NO | — |
| ProductId | int | NO | — |
| AddedUtc | datetime2 | NO | — |

---

## Relationship overview

High-level grouping of the 30 tables.

### Identity and accounts

- `Users` - accounts (customers, vendors, admins).
- `UserRoles` - role membership (User / Vendor / Admin).
- `Addresses` -> `Users`.
- `Carts` -> `Users` (nullable, SET NULL on delete); `CartItems` -> `Carts`, `ProductVariants`.
- `WishlistItems` -> `Users`, `Products`.

### Catalogue

- `Categories` (self-referencing via `ParentId`) -> `Shops`.
- `Products` -> `Shops`, `Categories`, `TaxClasses`.
- `ProductVariants` -> `Products`.
- `ProductImages` -> `Products`.
- `ShopListings` -> `Shops`, `ProductVariants` (a shop's listing of a variant).
- `TaxClasses` - tax rates.

### Orders and fulfilment

- `Orders` -> `Users` (nullable buyer), `Shops` (nullable).
- `OrderLines` -> `Orders`, `Products`, `ProductVariants` (snapshotted).
- `Payments` -> `Orders`; `PaymentEvents` -> `Payments`.
- `OrderShipments` -> `Orders`; `ShipmentEvents` -> `OrderShipments`.

### Returns (RMA)

- `ReturnRequests` -> `Orders`, `Shops`, `Users` (requester).

- `ReturnLines` -> `ReturnRequests`, `OrderLines`.

### Reviews

- `Reviews` -> `Products`, `Users`, `OrderLines` (verified purchase).
- `ReviewVotes` -> `Reviews`, `Users`.

### Marketplace

- `Shops` -> `Users` (owner).
- `ShopUsers` -> `Shops`, `Users`.
- `Coupons` -> `Shops` (nullable, shop-scoped).
- `CouponUsages` -> `Coupons`, `Users`, `Orders`.

### Vendor money

- `VendorLedgerEntries` -> `Shops`, `Orders` (nullable).
- `VendorPayouts` -> `Shops`.

### Messaging and observability

- `EmailMessages` - transactional outbox (no FKs; loose references).
- `AuditLogs` - administrative trail (loose references: entity type + id, actor name).
- `Log` - application log sink.

---

## Key data flows

How the tables participate in the store's four money-touching journeys.

### 1. Purchase (cart to paid order)

```
CartItems/Carts (browse-time basket, cookie- or user-owned)
|
v Checkout prices the cart fresh, applies the coupon, commits stock
Orders (totals snapshot, CouponId, IdempotencyKey, CollectionStatus)
|
+--< OrderLines (per-line snapshots of Products/ProductVariants)
|
+--< StockMovements (Reason=Sale, negative quantities)
|
+--< Payments (Pending charge row)
|
+--< PaymentEvents (signed gateway events, unique per Provider+EventId)
```

When the gateway confirms capture - or the carrier collects cash on delivery, or staff mark a counter order paid - one `VendorLedgerEntry (Sale)` is written, idempotent per order, crediting gross and withholding commission. When the order moves to Sold/Shipped/Delivered the status column moves with it and an `EmailMessages` row (order-confirmation, order-shipped) is queued.

### 2. Return (RMA) with its refund

```
Orders --< ReturnRequests --< ReturnLines --< OrderLines
| |
| +--< StockMovements (Reason=Return, positive)
|
+--< VendorLedgerEntries (Reversal, ReturnRequestId, proportional)
+--< Payments (Refunded row: refunded money back to the shopper)
+--< AuditLogs (return.approved/rejected/received, refund.issued/failed)
+--< EmailMessages (return-decided, return-refunded)
```

The service keeps one open return per order, validates the lifecycle Requested -> Approved -> Received -> Refunded (with Rejected/Cancelled exits), recomputes the refund from `ReturnLines` + order discount + tax, caps it by the ledger remainder and by `GrandTotal - RefundedTotal`, and restocks the received goods into `ShopListings`.

### 3. Settlement (paying a vendor)

```
VendorLedgerEntry (Sale rows, SettledUtc is null) = what the shop is owed
|
v admin settles at /Admin/Payouts
VendorPayouts (reference PAY-..., period, entry count, gross/commission/net, Status)
```

A payout caps the covered entries with SettledUtc and its own reference but writes no earnings, so a statement is reproducible from the ledger alone. A `Failed` payout (bank refused) or a `Reversed` one (recorded in error) stamps `FailedUtc`/`ReversedUtc`, `HandledBy` and a reason, and releases the covered entries back to unsettled - the balance is derived again, nothing is edited. Both actions are audited and email the shop's contact.

### 4. Notifications (outbox, not SMTP)

Every money-touching action above also writes `EmailMessages`; the dispatcher (background service) hands each to the transport and stamps `Sent` / `Failed` / `Skipped`. A DedupeKey per (modelled fact) means a retried request cannot mail twice.

---

## Status and enum columns

Several `int` columns are enum values from `Domain/Enums.cs` (and the neighbouring domain files). The codes below are what the SQL actually stores.

**Orders.Status** (`OrderStatus`)

| Code | Meaning |
|---|---|
| 0 | Pending payment |
| 1 | Paid |
| 2 | Shipped |
| 3 | Delivered |
| 4 | Cancelled |
| 5 | Refunded |

Legal transitions are validated in the service: Pending -> Paid -> Shipped -> Delivered with Cancelled/Refunded as exits; nothing can be re-opened from Cancelled/Refunded.

**Payments.Status** (`PaymentStatus`): 0 Pending, 1 Succeeded, 2 Failed, 3 Refunded.

**Payments.Method** and `Orders.PaymentMethod` (`PaymentMethod`): 0 CashOnDelivery, 1 Card, 2 CounterPayment.

**Orders.CollectionStatus** (`CollectionStatus`, cash-on-delivery money): 0 NotApplicable (prepaid), 1 AwaitingCollection, 2 Collected, 3 CollectionFailed, 4 Refunded.

**Orders.ShippingStatus** (`ShipmentStatus`): 0 Draft, 1 Created, 2 PickedUp, 3 InTransit, 4 OutForDelivery, 5 Delivered, 6 Failed, 7 Returned, 8 Cancelled.

**StockMovements.Reason** (`StockMovementReason`): 0 InitialStock, 1 Restock, 2 Sale, 3 Return, 4 Adjustment, 5 Cancellation.

**Coupons.DiscountType** (`DiscountType`): 0 Percentage, 1 FixedAmount.

**Shops.Status** (`ShopStatus`): 0 PendingApproval, 1 Approved, 2 Rejected, 3 Suspended, 4 Closed.

**Shops.CommissionPercent, ReviewShop rating etc.** are decimal fractions/plain values.

**VendorLedgerEntries.Type** (`VendorLedgerEntryType`): 0 Sale, 1 Reversal, 2 Adjustment, 3 Payout. `SettledUtc`, `PayoutId`, `PayoutReference` carry the settlement stamp.

**VendorPayouts.Status** (`PayoutStatus`): 0 Paid, 1 Failed, 2 Reversed.

**ReturnRequests.Status** (`ReturnStatus`): 0 Requested, 1 Approved, 2 Rejected, 3 Received, 4 Refunded, 5 Cancelled.

**Reviews.Status** (`ReviewStatus`): 0 Pending, 1 Published, 2 Rejected.

**EmailMessages.Status** (`EmailStatus`): 0 Queued, 1 Sent, 2 Failed, 3 Skipped.

**UserRoles.RoleName** / **ShopUsers.Role** are short strings (User/Vendor/Admin and Owner/Manager/Fulfilment) stored as plain text.

### What is derived, what is stamped

- The shop's *outstanding balance* is always the sum of the **unsettled** `VendorLedgerEntries`,

computed at query time - never a stored balance column.

- `Orders.RefundedTotal`, `ReturnRequests.RefundAmount`, `VendorPayouts` totals, and

`ProductVariants.Quantity` (stock) are all *consistent with the ledger/movement rows* behind them by convention of the code, and the audit trail records every mutation.

---

## Uniqueness and enforcement of the rules

There are **no** `UNIQUE` **constraints** on the app tables; uniqueness is enforced by unique indexes (from EF Core relationships / configured in `OnModelCreating`), and, where a rule spans rows, in the service layer rather than the DB. The important ones:

| Table | Unique index | Why |
|---|---|---|
| `Orders` | `IdempotencyKey` | A retried checkout POST cannot create a second order. |
| `Orders` | `OrderNumber` | Human-facing `ORD-...` reference. |
| `PaymentEvents` | `Provider + EventId` | A redelivered webhook is stored once; capture decisions stay idempotent. |
| `VendorPayouts` | `Reference` | Human-facing `PAY-...` reference. |
| `Products` | `ShopId + Slug` | A shop's catalogue URLs are unambiguous. |
| `Categories` | `ShopId + Slug` | Same for category trees. |
| `ProductVariants` | `Sku` | No two buyable SKUs collide store-wide. |
| `Coupons` | `Code` | One behaviour per code. |
| `TaxClasses` | `Name` | One rate per class name. |
| `Users` | `NormalizedEmail` | Sign-in lookup. |
| `UserRoles` | `UserId + RoleName` | A user holds a role once. |
| `ShopUsers` | `ShopId + UserId` | One membership row per (shop, user). |

| Table | Unique index | Why |
|---|---|---|
| `ShopListings` | `ShopId + ProductId` | One listing per shop per product. |
| `Carts` | (none) | At most one per user by convention in the service. |
| `CartItems` | `CartId + ProductVariantId` | Adding the same variant twice increments a quantity, not a row. |
| `Reviews` | (none) | "One review per shopper per product" is enforced in the service, not the schema. |
| `ReviewVotes` | `ReviewId + UserId` | One vote per user per review. |
| `WishlistItems` | `UserId + ProductId` | A wishlist entry appears once. |
| `VendorLedgerEntries` | (none by design) | "Exactly once" crediting is enforced by querying-then-inserting inside a transaction; the safest approach without a unique key that cannot filter `if not exists` cleanly. |

## Design notes

- **Money is** `decimal(18,2)` and the arithmetic is done in C# (`Math.Round` half-away-from-zero

for commission) - SQLite, the second supported provider, cannot aggregate `decimal` in SQL, so money sums cross the wire as rows and are summed in memory.

- **Snapshots, not references, for history.** `OrderLines` stores product/variant names, SKU and

unit price at purchase time; therefore an edited product or a renamed variant cannot rewrite a past order.

- **Soft delete for the catalogue.** `Products.IsDeleted` hides without destroying; sold

variants are deactivated (`IsActive = 0`) so past orders and restocks still resolve.

- **Append-only ledgers.** `VendorLedgerEntries`, `StockMovements` and `AuditLogs` are never

edited after write; corrections are new rows, which makes the money and stock trails replayable.

- **UTC everywhere.** Every timestamp column is `datetime2` in UTC (`...Utc` by convention).
- **Loose references for the trail.** `AuditLogs.EntityType + EntityId` and

`EmailMessages.UserId/OrderId` are ordinary columns, not foreign keys with cascade rules - a deleted account must never rewrite what the audit or the outbox said at the time.

- **Idempotency at the boundaries.** `Orders.IdempotencyKey` (checkout), `PaymentEvents`

provider+event id (webhooks), and `EmailMessages.DedupeKey` (notifications) survive retries; ledger crediting is guarded by the service layer as described above.

- **Provider portability.** The model is EF Core-first and runs on SQLite or SQL Server; new

columns arrive through guarded `ADD COLUMN` DDL in `Data/` (the `ReviewSchema.cs`, `EmailSchema.cs`, `CouponSchema.cs`, `PaymentSchema.cs`, `VendorSchema.cs`, `ReturnsSchema.cs` family; there are no migrations), which is why this list stays honest: it reflects what the store runs on.

## Data volume and analysis (live demo instance)

The figures below are row counts from the running MVStores database at the time of writing. They give a sense of which tables carry the bulk of the data and which are configuration/lookup tables.

| Table | Rows | | Table | Rows |
|---|---|---|---|---|
| Addresses | 2 | | Products | 18 |
| AuditLogs | 13 | | ReturnLines | 1 |

| Table | Rows | | Table | Rows |
|---|---|---|---|---|
| CartItems | 1 | | ReturnRequests | 1 |
| Carts | 1 | | ReviewVotes | 0 |
| Categories | 15 | | Reviews | 225 |
| CouponUsages | 0 | | ShipmentEvents | 1 |
| Coupons | 3 | | ShopListings | 18 |
| EmailMessages | 7 | | ShopUsers | 2 |
| Log | 743 | | Shops | 2 |
| OrderLines | 3 | | StockMovements | 5 |
| OrderShipments | 1 | | TaxClasses | 1 |
| Orders | 2 | | UserRoles | 3 |
| PaymentEvents | 1 | | Users | 2 |
| Payments | 3 | | VendorLedgerEntries | 3 |
| ProductImages | 54 | | VendorPayouts | 5 |
| ProductVariants | 28 | | WishlistItems | 0 |

### Observations

- **Log dominates** (743 rows) - the database log sink; every application line lands here. Retention is off by default, so this grows until Logging:Database:RetentionDays is set.
- **Reviews is the next largest (225)** - the demo seeds a large review set for the marketing/reputation features.
- **ProductImages (54)** and **Products (18) / ProductVariants (28)** - the catalogue is small; image rows outnumber products because of galleries.
- **AuditLogs (13)** - the administrative trail already records the live smoke-test actions (returns, refunds, payouts, shop suspend/reinstate).
- **VendorPayouts (5)** - includes the seeded payout plus the live mark-failed / reverse / re-settle cycle.
- **ReturnRequests (1) / ReturnLines (1)** - the single end-to-end RMA exercised in the smoke test.
- **Empty tables by design**: CouponUsages, ReviewVotes, WishlistItems - no redemptions/votes/wishlist entries in the demo data.
- **Configuration/lookup tables are tiny**: TaxClasses (1), Shops (2), Users (2), Categories (15).

### Where the weight is

Storage is dominated by Log, then Reviews and ProductImages. Everything else is small enough that the whole store fits comfortably in a single SQL Server database or a SQLite file; the Data/*Schema.cs guarded DDL and the in-memory EF test provider both assume that scale.